

UD3. Fundamentos de Contenedores y Docker

Los contenedores son unidades de software que empaquetan el código y todas sus dependencias, lo que permite que una aplicación se ejecute de manera rápida y confiable en diferentes entornos informáticos. Esta tecnología proporciona una abstracción de la infraestructura subyacente y facilita la portabilidad entre entornos de nube y sistemas operativos. Comparados con las máquinas virtuales, **los contenedores comparten el sistema operativo del host y no necesitan incluir un sistema operativo completo en cada contenedor, lo que los hace mucho más ligeros y rápidos de iniciar.**

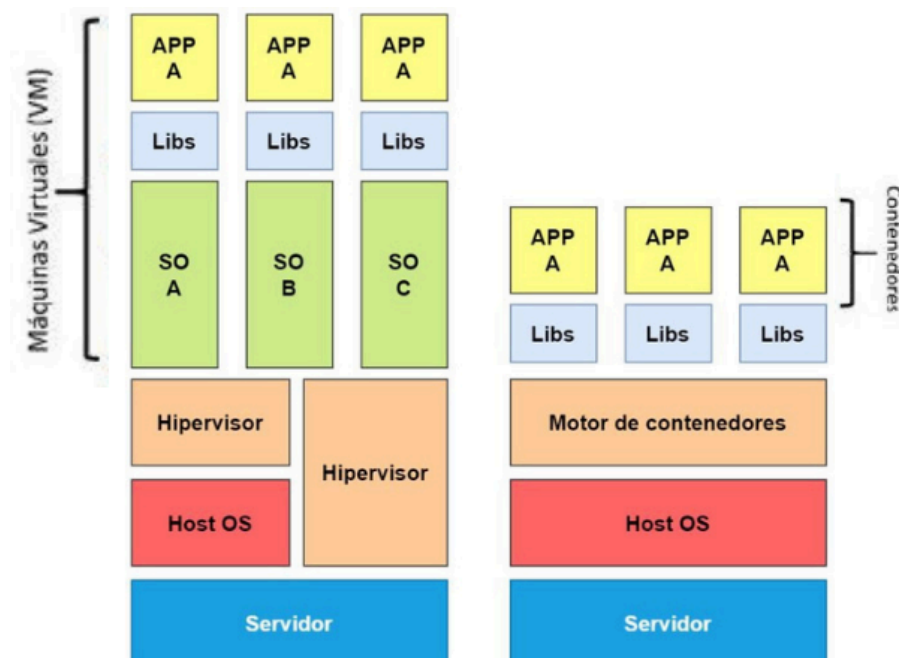
Docker, en particular, es una plataforma de código abierto que ha popularizado el uso de contenedores en el desarrollo de software y operaciones de TI. Docker permite a los desarrolladores crear, desplegar y ejecutar aplicaciones en contenedores que se comportan de manera consistente en cualquier entorno, reduciendo así los problemas clásicos de "funciona en mi máquina". La herramienta utiliza una sintaxis sencilla para definir y gestionar contenedores mediante **Dockerfiles** y **comandos** que simplifican la creación y el manejo de imágenes y contenedores de aplicaciones.

El ecosistema de Docker no solo incluye la herramienta principal, sino también un registro de contenedores denominado **Docker Hub**, donde los usuarios pueden almacenar y compartir imágenes de contenedores públicamente o en repositorios privados. Además, Docker se integra con diversas plataformas de orquestación de contenedores como Kubernetes, lo que facilita la gestión de despliegues a gran escala en entornos de producción. Esta integración y facilidad de uso han hecho de Docker una opción esencial para las empresas que adoptan prácticas de DevOps y desarrollo ágil.

Licencia:

Creative Commons Atribución-NoComercial-CompartirIgual (CC BY-NC-SA 4.0)

1. Contenedores



A pesar de que los equipos son cada vez más potentes, la ejecución de diferentes máquinas virtuales sobre un mismo equipo no es la solución más óptima ya que se ha de virtualizar todo el hardware repetidamente. Además los tiempos de arranque son considerables en el caso de servidores de aplicaciones o sistemas críticos.

Es ahí donde surge la **virtualización a nivel de sistema operativo** que se basa en añadir una capa de virtualización sobre el mismo sistema operativo para tener instancias aisladas del espacio de usuario (espacio de memoria, recursos de E/S, almacenamiento, librerías) realizando las llamadas al núcleo del sistema como, por ejemplo, solicitar el uso de un periférico, denominándose a esta técnica **virtualización basada en contenedores** y a la tecnología que permite esto: cgroups y namespaces.

Estas diferencias implican que sobre el mismo hardware es posible ejecutar muchos más contenedores que máquinas virtuales. Además, el tiempo de arranque se ve reducido notablemente. Por ejemplo, en la plataforma de virtualización Proxmox se necesita mínimo 1 GB de memoria RAM para crear una máquina virtual con Ubuntu Server, en cambio, un contenedor únicamente necesita 512 MB y en un uso típico (servidor DNS, DHCP, cortafuegos) no suele consumir más de 400 MB. Además es posible mover de forma sencilla los contenedores entre servidores, por ejemplo, ante una actualización del anfitrión.

Existen diversas implementaciones de contenedores destacando:

- **LXC: Linux containers.** Proporciona diferentes herramientas para la gestión de los contenedores.
- **LXD: Linux Container Daemon**, basado en **LXC**, posee un demonio que gestiona los diferentes contenedores y proporciona una API REST para la gestión de los mismos. También proporciona una interfaz de comandos para la gestión de los contenedores. Posee instantáneas, es posible descargar imágenes de contenedores y establecer perfiles según el uso del contenedor.
- **Docker.** Similar a los anteriores, el sistema aísla al software del contenedor de sistema operativo host, como árbol de procesos, red, usuario o sistemas de archivos, además de limitar el acceso a los recursos hardware (memoria, E/S o la red). Proporciona una **CLI**, así como una **API REST**. Posee un repositorio en el que almacenar y descargar imágenes, además es posible definir imágenes personalizadas.



Q dotCloud, Inc. Logo of Docker, a Linux container engine. (Licencia Apache 2.0)

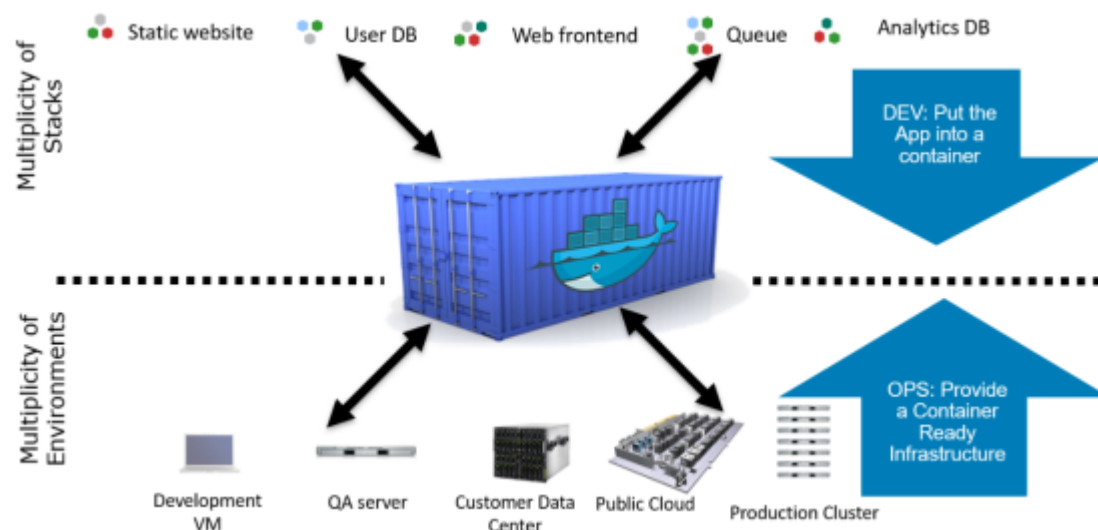
Existen otras implementaciones de contenedores pero las 3 anteriores son las más utilizadas en la actualidad.

El nombre **docker** está inspirado en el concepto de "dock" como un puerto o punto de conexión. Docker es una plataforma que permite empaquetar aplicaciones y sus dependencias en contenedores, asegurando que puedan "atracar" o ejecutarse en cualquier sistema de manera eficiente y consistente.

Relación del nombre con su función

- **Muelle de contenedores:** Docker usa el concepto de **contenedores** (containers), que en el mundo físico se cargan y descargan en muelles (docks). Este paralelo es perfecto para explicar cómo Docker gestiona contenedores de software: como si fueran contenedores de carga que se transportan fácilmente entre diferentes puertos (sistemas operativos o servidores).
- **Portabilidad y consistencia:** Así como un contenedor de carga no necesita que su contenido se adapte a cada muelle, un contenedor Docker no necesita que su aplicación se adapte al entorno donde se ejecuta.

En resumen, el nombre **Docker** refuerza la idea de portabilidad, flexibilidad y estandarización, tal como los contenedores y muelles en el transporte marítimo.

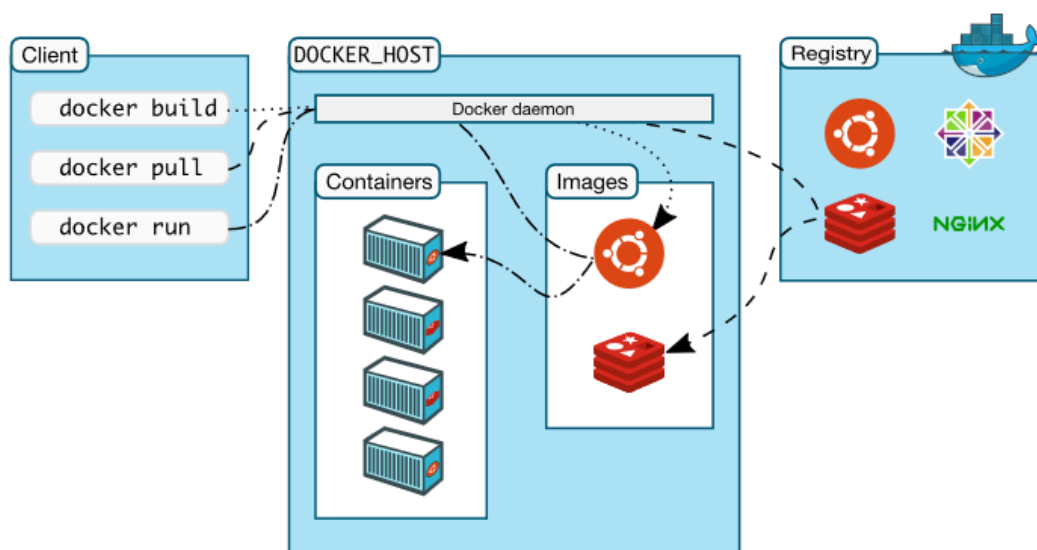


2. Docker

Docker es el sistema de virtualización con contenedores más utilizado en la actualidad y posee una gran cantidad de repositorios e imágenes, además de poder crear y personalizar imágenes propias en función de las necesidades.

Debemos diferenciar entre:

- ❑ **Repositorio:** es el lugar donde se encuentran la imágenes y que normalmente es una ubicación remota.
- ❑ **Imagen:** es la plantilla en la que no se puede escribir y en la que se definen las instrucciones para la creación del contenedor.
- ❑ **Contenedor:** Instancia ejecutable de una imagen y que ejecuta la aplicación o servicio.



Debes conocer



Descripción

Ventajas de Docker o de los contenedores (en resumen):

- 1. Portable:** Nos abstraemos del sistema operativo, lo que facilita mover aplicaciones entre diferentes entornos sin modificaciones.
- 2. Inmutable:** Empaqueta todas las dependencias necesarias, asegurando que el entorno de ejecución no cambie.
- 3. Ligero:** Al usar **cgroups** y **namespaces** de Linux, los contenedores funcionan como procesos aislados dentro del sistema operativo.

Ventajas adicionales:

- ❑ **Configuración extremadamente simple:** Se logra mediante un script llamado **Dockerfile**, que permite definir el entorno de manera declarativa.
- ❑ **Velocidad:** El arranque de un contenedor es muy rápido, comparable a ejecutar un programa directamente en el sistema operativo.

2.1. Instalación

Tenemos versiones de docker para **Windows** (necesita **WSL2**), **Linux** y **Mac**. La instalación pone a nuestra disposición una serie de comandos **CLI** para interactuar con el **API** del motor de docker que permite gestionar elementos para la red, volúmenes, construcción, distribución u orquestación de contenedores entre otros.

Docker se puede encontrar en: <https://docs.docker.com/engine/install/>

Una vez instalado se puede ir a la línea de comandos y acceder al **Docker CLI o cliente de Docker** con el comando:

```
docker
```

Apareciendo el listado de comandos disponible (CLI):

```
Management Commands:
app*      Docker App (Docker Inc., v0.9.1-beta3)
builder   Manage builds
buildx*   Build with BuildKit (Docker Inc., v0.5.1-docker)
compose*  Docker Compose (Docker Inc., 2.0.0-beta.1)
config    Manage Docker configs
container Manage containers
context   Manage contexts
image     Manage images
manifest  Manage Docker image manifests and manifest lists
network   Manage networks
node      Manage Swarm nodes
plugin    Manage plugins
scan*     Docker Scan (Docker Inc., v0.8.0)
secret    Manage Docker secrets
service   Manage services
stack     Manage Docker stacks
swarm     Manage Swarm
system    Manage Docker
trust     Manage trust on Docker images
volume    Manage volumes

Commands:
attach    Attach local standard input, output, and error streams to a running container
build     Build an image from a Dockerfile
commit    Create a new image from a container's changes
cp        Copy files/folders between a container and the local filesystem
create    Create a new container
diff      Inspect changes to files or directories on a container's filesystem
events    Get real time events from the server
exec      Run a command in a running container
export    Export a container's filesystem as a tar archive
```

Ejercicio

Instalación de Docker  en Ubuntu 



Descripción

```
Bash script

#!/bin/bash

# sacado de https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Comprobación de su correcta instalación
sudo systemctl status docker containerd
docker --version

# Correr el primer contenedor de prueba
sudo docker run hello-world
```



Para saber más

Sí, puedes instalar Docker Engine en Windows aunque esté diseñado para sistemas Linux. Su funcionamiento en Windows por tanto requiere un entorno Linux simulado, por ello usar [Docker Desktop](#) o [WSL 2](#) es la forma más estable y soportada para trabajar con Docker en Windows. En el caso de Docker Desktop para Windows, se instala una interfaz gráfica en la que se pueden ejecutar algunos de los comandos anteriores, realizar algunas configuraciones y administrar las imágenes y contenedores instalados. Es posible instalar interfaces gráficas para la gestión de Docker, en el  [siguiente artículo se listan algunas de ellas](#): <https://gcore.com/blog/top-gui-for-docker/>

2.2. Imágenes y contenedores

Imágenes

- Una imagen de Docker funciona como un modelo base para crear contenedores. Consiste en un sistema de archivos estructurado en capas que no se pueden modificar (Union File System).
- El propósito principal de las imágenes es empaquetar y distribuir el software que se ejecutará en los contenedores.
- Se pueden obtener imágenes de un Registro de Docker `docker pull` o crearlas directamente en tu entorno local `docker build`.
- Es importante asegurarse de que las imágenes descargadas sean confiables; para ello, lo ideal es usar las imágenes oficiales de Docker.

- Un ejemplo es **BusyBox** es una imagen minimalista que incluye herramientas de Linux, útiles para realizar diversas tareas dentro de los contenedores.
- También se usa Linux **Alpine** como imagen base para contenedores dado que es una versión ligera de *libc* que incluye las herramientas de BusyBox. Esto permite crear imágenes muy pequeñas, que arrancan rápidamente y ofrecen un alto nivel de seguridad.

Contenedores

- El propósito principal de los contenedores es ejecutarse en un Host. Las formas más comunes de iniciar un contenedor utilizando Docker CLI son:
 - Crear el contenedor con `docker create` y luego iniciarlo con `docker start`.
 - Ejecutarlo directamente con `docker run`.
- Cuando se ejecuta, el contenedor agrega una capa de lectura/escritura sobre las capas inmutables de la imagen. De manera predeterminada, los datos de esta capa desaparecerán cuando el contenedor deje de existir.
- Los contenedores pueden realizar tareas que finalizan o permanecer en ejecución continua como servicios (por ejemplo, servidores web o bases de datos). Un contenedor puede tener diferentes estados: **created** (creado). **restarting** (reiniciando). **running** (en ejecución). **removing** (eliminándose). **paused** (pausado). **exited** (detenido). **dead** (muerto).
- Cuando un contenedor se inicia, es posible ejecutarlo en segundo plano (`-d`) o vincular su salida a la terminal. Si el contenedor se ejecuta en segundo plano, puedes acceder a los registros con el comando `docker log`.

Ahora que tenemos Docker instalado podemos, bien crear imágenes y contenedores o usar un repositorio de imágenes como [DockerHub](#). Este repositorio dispone de cuentas gratuitas y de pago, y permite subir imágenes propias para su posterior uso. Por ejemplo, si queremos instalar una imagen de la conocida tienda web magento, iremos a la página web del repositorio y buscaremos la palabra magento o usando el comando:

```
docker search magento
```

```
root@pedro:/home/pedro# docker search magento
```

NAME	DESCRIPTION	STARS	OFFICIAL
AUTOMATED			
alexcheng/magento2	Docker image for Magento 2	169	
[OK]			
alexcheng/magento	Docker image for Magento	103	
[OK]			
magento/magento2devbox-web	Official Web container with Magento instance	62	
[OK]			
magento/magento-cloud-docker-php	Magento Cloud Docker - PHP	41	
[OK]			

Una vez que hemos localizado la imagen, ejecutaremos el comando:

```
docker pull alexcheng/magento2
```

```
root@pedro:/home/pedro# docker pull alexcheng/magento2
Using default tag: latest
latest: Pulling from alexcheng/magento2
8ee29e426c26: Pulling fs layer
6e83b260b73b: Pulling fs layer
e26b65fd1143: Pulling fs layer
40dca07f8222: Pulling fs layer
1400...: Pulling fs layer
```


A partir de este momento, ya podemos crear contenedores a partir de la imagen descargada. Para comprobar las imágenes locales:

```
docker images
```

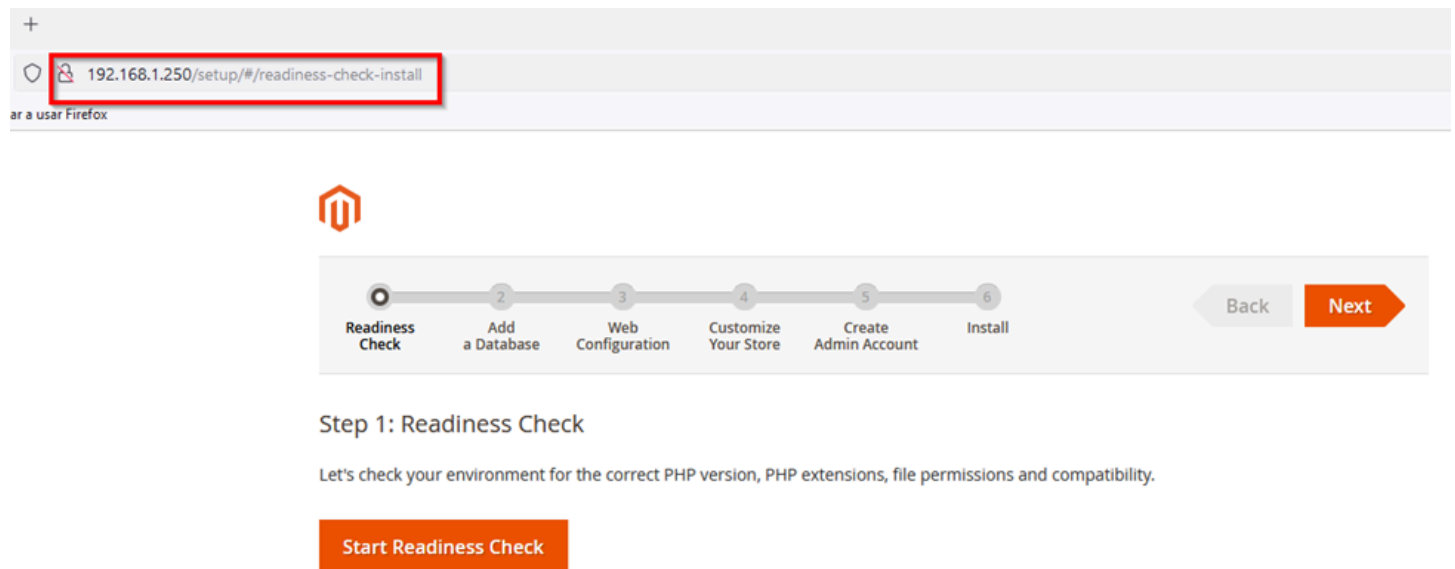
Para iniciar un contenedor ejecutaremos:

```
docker run -p [host port]:[container port] [nombre_imagen ]
```

en el que el parámetro **-p** indica el mapeo de puerto de la máquina anfitriona y del contenedor, en este caso:

```
docker run -p 80:80 alexcheng/magento2
```

Al entrar en la IP o URL de la máquina anfitriona se puede ver el instalador de Magento para iniciar el proceso.



Para parar el contenedor ejecutar:

```
docker stop [nombre_contenedor]
```

Es posible ver los contenedores de la máquina con el comando

```
docker container ls -all
```

```
root@pedro:/home/pedro# docker container ls -all
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
7079711cb350   magento/magento2devbox-web         "/usr/local/bin/entr..." About an hour ago Exited (137) Ab
```

o con su alias `docker ps` : <https://docs.docker.com/reference/cli/docker/container/ls/>

unidad3 imagen docker



Descripción

Comandos



Para saber más

Es muy recomendable que consultes el listado completo de comandos y opciones disponibles en Docker, a través de las páginas de documentación de su web oficial:

<https://docs.docker.com/engine/reference/run/>

2.3. Persistencia de datos

Volumenes

Los contenedores no tienen acceso al sistema de archivos y son simplemente un conjunto de procesos y librerías aislados del resto de los elementos del sistema operativo anfitrión. Al eliminar un contenedor, también se eliminan los datos que contiene.

Para solucionar este problema la opción más utilizada, es la de creación de **volúmenes**  y asignación a los contenedores. Un volumen puede estar ubicado en el disco del Host donde se está ejecutando Docker o en discos de red dentro de un centro de datos, algo común en entornos de Cloud Computing.

Los principales usos de los volúmenes incluyen:

- ☐ Compartir datos entre diferentes contenedores.
- ☐ Realizar copias de seguridad.
- ☐ Transferir contenedores entre Hosts que usan distintas plataformas.

- ❑ **Almacenamiento en entornos Cloud:** Los volúmenes de Docker permiten compartir datos entre varios contenedores o mantenerse separados si no están asignados a ninguno.
- ❑ **Gestión de volúmenes con Docker:**
 - Crear nuevos volúmenes con el comando `docker volume create`.
 - Ver los volúmenes existentes con `docker volume ls`.
 - Eliminar volúmenes que ya no se necesitan con `docker volume rm`.
- ❑ También se pueden crear volúmenes al lanzar un contenedor usando el flag `-v` en el comando `docker run`.
- ❑ Los volúmenes ya creados pueden ser reutilizados en otros contenedores.
- ❑ Además de los volúmenes, los contenedores pueden montar directorios del sistema de archivos del host (usando *bind mounts*).

Sintaxis:

```
docker volume create [nombre_volumen]
```

Para listar los volúmenes existentes:

```
docker volume ls
```

Y para ver los detalles de alguno de ellos:

```
docker volume inspect [nombre]
```

```
root@pedro:/home/pedro# docker volume create ejemplovolumen
ejemplovolumen
root@pedro:/home/pedro# docker volume ls
DRIVER      VOLUME NAME
local      ejemplovolumen
root@pedro:/home/pedro# docker volume inspect ejemplovolumen
[
  {
    "CreatedAt": "2021-07-04T15:02:04Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/ejemplovolumen/_data",
    "Name": "ejemplovolumen",
    "Options": {},
    "Scope": "local"
  }
]
```

Por último, si queremos borrar un volumen usaremos el [comando](#):

```
docker volume rm [nombre]
```

Para “montar” el volumen en el árbol de directorios del contenedor usaremos el comando **run** con el parámetro **mount** en el que se indica el volumen con **source** y, con **target**, el punto de montaje dentro del contenedor.

Un ejemplo con el volumen anterior:

```
docker run --mount source=ejemplovolumen,target=/var/lib/mysql
#o abreviado:
docker run -v ejemplovolumen:/var/lib/mysql
```



Ejercicio

unidad3 volumen docker



Descripción

Comandos

2.4. Dockerfile

Es posible la ejecución de parámetros en el contenedor tanto al iniciarlo como una vez en ejecución usando los comandos:

docker run

docker exec

respectivamente. Por ejemplo para realizar un **ls** dentro de un contenedor ya iniciado con las opciones típicas de terminal interactiva **-it** :

```
root@pedro:/home/pedro# docker exec -it confident_poitras ls -l
total 1224
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 app
-rw-rw--- 1 www-data www-data 234 Apr  2 2019 auth.json.sample
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 bin
-rw-rw--- 1 www-data www-data 434981 Apr  2 2019 CHANGELOG.md
-rw-rw--- 1 www-data www-data 2257 Mar 15 2019 composer.json
-rw-rw--- 1 www-data www-data 706546 Apr  2 2019 composer.lock
-rw-rw--- 1 www-data www-data 650 Apr  2 2019 COPYING.txt
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 dev
drwxrwx--- 1 www-data www-data 4096 Jul  5 15:25 generated
-rw-rw--- 1 www-data www-data 57 Apr  2 2019 grunt-config.json.sample
-rw-rw--- 1 www-data www-data 2994 Apr  2 2019 Gruntfile.js.sample
-rw-rw--- 1 www-data www-data 1370 Apr  2 2019 index.php
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 lib
-rw-rw--- 1 www-data www-data 10376 Apr  2 2019 LICENSE_AFL.txt
-rw-rw--- 1 www-data www-data 10364 Apr  2 2019 LICENSE.txt
-rw-rw--- 1 www-data www-data 5625 Apr  2 2019 nginx.conf.sample
-rw-rw--- 1 www-data www-data 1416 Apr  2 2019 package.json.sample
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 phpserver
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 pub
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 setup
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 update
drwxrwx--- 1 www-data www-data 4096 Jul  5 15:49 var
drwxrwx--- 1 www-data www-data 4096 Apr  2 2019 vendor
```

Q Pulsar para ampliar

Si tenemos que estar ejecutando cada uno de los **comandos** en el contenedor para configurarlo, rápidamente veremos que es una tarea tediosa. Además, tenemos que buscar cómo adaptar una imagen base a las necesidades del proyecto.

Para crear una imagen personalizada, se utiliza el fichero **Dockerfile**, que contiene comandos que se ejecutan al instanciar la imagen y, estos comandos, se encuentran precedidos de otros que indican ciertas acciones:

- ❑ **FROM [imagen]** Indica la imagen base.
- ❑ **RUN [comando] [parámetros]** Ejecuta comandos durante la construcción de la imagen, útil para instalar paquetes necesarios.
- ❑ **CMD [comando] [parámetros]** define el programa por defecto que se ejecutará, pero puede ser modificado al iniciar el contenedor con `docker run` .
- ❑ **ENTRYPOINT:** Indica el comando y los parámetros a ejecutar al iniciar el contenedor. Configura el contenedor para ejecutarse como un comando específico.
- ❑ **ADD** Posee dos formatos:
 - **ADD [--chown=<user>:<group>] <src>... <dest>**
 - **ADD [--chown=<user>:<group>] ["<src>",... "<dest>"]**

Por supuesto **chown solo es aplicable en contenedores Linux**. La segunda forma simplemente añade un conjunto de ficheros/directorios de forma local y/o remota. En caso de añadir ficheros comprimidos es capaz de reconocerlos y descomprimir.

- ❑ **COPY** la documentación oficial recomienda este comando antes que **ADD** en ficheros normales (no comprimidos).
- ❑ **EXPOSE <port> [<port>/<protocol>...]**. Permite establecer los puertos en lo que el contenedor se encuentra escuchando.
- ❑ **ENV <key>=<value>** Establece un valor a variables de entorno. **Ventajas:** Flexibilidad para actualizar sin modificar el código. Específicas del entorno.
- ❑ **LABEL <key>=<value>** Añade **metainformación** a la imagen.
- ❑ **VOLUME** Crea un volumen y lo monta en el directorio indicado. Veamos un ejemplo de la documentación oficial:

```
1 FROM ubuntu
2
3 RUN mkdir /myvol
4
5 RUN echo "hello world" > /myvol/greeting
6
7 VOLUME /myvol
```

□ **WORKDIR** Establece el directorio en el que se ejecutan el resto de comandos.

Una vez creado el **Dockerfile** se ejecuta este comando en el mismo directorio en el que se encuentra el fichero:

```
docker build -t nombre_imagen .
```

Con el **.** le indicamos al cliente Docker donde debe buscar el Dockerfile para montar la imagen (también se puede especificar con la opción **-f**). Veamos un ejemplo de fichero Dockerfile basado en Debian en el que se instala Apache y se copia en la carpeta por defecto un fichero del anfitrión.

```
1 FROM debian
2
3 RUN apt update
4
5 RUN DEBIAN_FRONTEND="noninteractive" apt install apache2 -y
6
7 EXPOSE 80
8
9 COPY ./proyecto/* /var/www/html
10
11 CMD ["/bin/bash"]
```



Tarea

unidad3 dockerfile docker





Para saber más

Buenas prácticas para la creación de ficheros Dockerfile.

Este documento cubre las mejores prácticas y métodos recomendados para crear imágenes eficientes:

https://docs.docker.com/develop/develop-images/dockerfile_best-practices/.

Docker crea imágenes automáticamente leyendo las instrucciones de un archivo **Dockerfile** de texto que contiene todos los comandos, necesarios para crear una imagen determinada. El formato específico y un conjunto de instrucciones de **Dockerfile**, que puede encontrar en la referencia de la documentación oficial.

Se puede encontrar información mucho más completa en la documentación oficial a cerca del comando

docker exec.

3. Práctica "Trasteando con Docker"



Ejercicios obligatorios

Vamos a poner a prueba los conceptos estudiados en la teoría de la unidad didáctica, que cubre los conceptos sobre contenedores y docker relacionados con:

- ☐ Instalación
- ☐ Imágenes
- ☐ Volúmenes
- ☐ Dockerfile



¿Qué te pedimos que hagas?

Requisitos Iniciales:

- Visiona los vídeos de la unidad.
- Realizar el examen online de la unidad, y consulta nuevamente las dudas que te surjan.
- Acceso a una terminal de Ubuntu con privilegios de administrador/root o -una vez instalado Docker Engine- pertenecer al grupo "docker".
- No olvides elaborar el documento explicativo.

Ejercicio 1 - Instalación de Docker

- Accede a una terminal de Ubuntu directamente o por ssh.
- Instala Docker Engine.
- Verifica que el servicio de docker corre correctamente y obtén la versión instalada.

Ejercicio 2 - Obtención de imágenes y generación de contenedores

- Busca en el repositorio de DockerHub la imagen oficial más reciente de JOOMLA (es un CMS similar a WordPress) y de MARIADB.
- Descarga en tu host las 2 imágenes encontradas; puedes usar la última versión con el tag "latest" aunque en producción, lo recomendable sería elegir una versión concreta.
- Crea una red para que los contenedores puedan comunicarse, con el comando: `docker network create joomla-network`
- Genera 2 contenedores en segundo plano que te permitan visualizar en el navegador web el punto de entrada de JOOMLA en el puerto 8088. Para ello lanza primero el de MARIADB, por ejemplo:


```
docker run --rm -d --name mariadb \
    -e MYSQL_ROOT_PASSWORD=org \
    -e MYSQL_DATABASE=joomla \
    -e MYSQL_USER=joomlauser \
    -e MYSQL_PASSWORD=org \
    --network joomla-network \
    mariadb:latest
```

Y luego el de JOOMLA, siguiendo el ejemplo:

```
docker run --rm -d --name joomla \
    -p 8088:80 \
    -e JOOMLA_DB_HOST=mariadb \
    -e JOOMLA_DB_USER=joomlauser \
    -e JOOMLA_DB_PASSWORD=org \
    -e JOOMLA_DB_NAME=joomla \
    --network joomla-network \
    joomla:latest
```

- Verifica que las imágenes y los contenedores son reconocidos por el demonio de Docker.

Ejercicio 3 - Generación de volúmenes

- Visualiza los volúmenes de Docker en tu host.
- Genera un volumen para cada contenedor, por ejemplo: joomla_datos, mariadb_datos.
- Detecta a qué directorio de tu host está vinculado cada volumen.
- Instancia un contenedor para MARIADB y otro para JOOMLA que respectivamente, monten los volúmenes creados en el punto anterior. En el primer caso para obtener persistencia en el directorio /var/lib/mysql (donde están las bases de datos) y en el segundo: /var/www/html (donde están los ficheros del servidor web con joomla).
- Verifica que si se crea un directorio en el contenedor JOOMLA, efectivamente se referencia en el volumen del host.

Ejercicio 4 - Dockerfile

- Genera un Dockerfile que automatice la obtención de la última imagen de JOOMLA, tratando de incluir aquí la especificación de las variables de entorno antes referidas. Especifica también un volumen que monte un directorio entre contenedor y host.
 - Genera un Dockerfile que automatice la obtención de la última imagen de MARIADB, tratando de incluir aquí la especificación de las variables de entorno antes referidas. Especifica también un volumen que monte un directorio entre contenedor y host.
 - Crea las imágenes a través de los archivos Dockerfile.
 - Genera los contenedores que te permitan visualizar JOOMLA en el navegador.
 - Verifica que los volúmenes funcionan correctamente. Para ello, puedes crear una carpeta, eliminar el contenedor y comprobar que los archivos persisten en el host o también, configurar JOOMLA desde el navegador, eliminar los contenedores y volver a arrancar ambos contenedores comprobando que la configuración inicial persiste.
-



Ejercicios opcionales: puede entregarse uno, varios, todos o ninguno.

1. Realiza el tipo test de la unidad 3 hasta que saques más de un 8 sobre 10 en la calificación. Manda una captura de pantalla insertada en el documento/plantilla cuando lo tengas.
2. Instala el Subsystem Linux para Windows o WSL con una maquina Ubuntu y prueba a ejecutar algún comando en ella.
3. Instala Docker Desktop en Windows (si optas por hacer este ejercicio tendrás que hacer antes el anterior).
4. Instala Portainer y Pruébalo.

3.1. Evaluación de la tarea

Criterios de evaluación implicados

¿Cómo valoro tu tarea?

Rúbrica de la tarea	
Ejercicio 1:	1,5 p.
Ejercicio 2:	2 p.
Ejercicio 3:	2,5 p.
Ejercicio 4:	2,5 puntos
Bibliografía web y Valoración personal	1 + 0,5 puntos
Ejercicios Opcionales y Retos:	hasta + 0.25 puntos extra cada uno
Ejercicios de Ampliación, Comentarios detallados	hasta + 1 punto extra cada uno

3.2. Para saber más

Glosario

Las palabras que debes entender en esta unidad relativa a Docker son:

- Un **ENTORNO** en el contexto del **desarrollo de software** se refiere a un espacio o sistema configurado con hardware, software, y datos específicos donde se realiza una actividad particular, como desarrollo, pruebas o producción, dependiendo del propósito o fase del ciclo de desarrollo.
- Un **HIPERVISOR** es un software o firmware que permite crear y gestionar máquinas virtuales (VMs) al abstraer los recursos físicos del hardware subyacente, como CPU, memoria y almacenamiento, y asignarlos de manera independiente a cada VM.
- El **HOST o anfitrión** es la máquina física que proporciona los recursos (CPU, memoria, almacenamiento y red) y ejecuta el hipervisor/motor de contenedores para crear y gestionar máquinas virtuales VMs/contenedores.
- La **INTEGRACIÓN CONTINUA** es una práctica de desarrollo donde los cambios en el código se integran frecuentemente en un repositorio compartido, con control de versiones y verificaciones automáticas para detectar errores temprano.
- Un **HASH** es un algoritmo matemático que toma un conjunto de datos de cualquier tamaño y los convierte en una cadena de caracteres con una longitud fija, independientemente del tamaño de los datos originales. La salida generada por este algoritmo también se denomina "hash".
- **SHA-256** es un algoritmo criptográfico de la familia SHA-2 que produce una salida de 256 bits, representado como una cadena hexadecimal de 64 caracteres. Destaca su resistencia a colisiones y dificultad para invertir el proceso.
- Un **PUNTO DE MONTAJE** es el directorío dentro del árbol de directorios del S.O. donde se integra/conecta otro sistema de ficheros almacenado en un dispositivo.
- **Registro de Docker:** Aplicación para almacenar y distribuir imágenes.
- **Docker Hub:** El registro más utilizado (hub.docker.com). También se pueden usar registros privados o configurar uno propio con la imagen `registry`.

Docker CLI

- **Otros comandos útiles:**
 - `docker top` : Procesos activos en un contenedor.
 - `docker stats` : Consumo de recursos.
 - `docker info` : Información del Docker Engine.
 - `docker inspect` : Detalles de elementos en Docker.
 - Subida de imágenes: Requiere autenticarse con `docker login` y usar `docker push`.
- **Importar/exportar contenedores:** Usa `docker export` y `docker import` para manejar sistemas de ficheros en formato `.tar`.

Docker Daemon

- **Servidor Docker Engine:** Exponer una API REST para comunicación con Docker CLI.
- **Sockets:** Utiliza `/var/run/docker.sock` un **archivo de socket Unix** utilizado por Docker para permitir la comunicación entre el cliente Docker (`docker CLI`) y el Docker Daemon (`dockerd`).

Herramientas gráficas

- **Opciones para evitar la línea de comandos:**

- Kitematic.
- Portainer.
- DockStation.

- Shipyard.

Buenas prácticas

□ Optimización:

- Usa un archivo `.dockerignore` para excluir archivos innecesarios.
- Agrupa las instrucciones `RUN` , `ADD` y `COPY` .

- **Multistage:** Construye imágenes en etapas, dejando solo lo esencial en la final.

Créditos



Por Fernando Barcina, profesor del Centro Integrado Público de Formación Profesional a Distancia de La Rioja

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0